SpecReel - Project Proposal (PA#1)

Course: CSC13114 | Team: Nguyễn Hoàng Gia Bảo - 23127157, Lương Linh Khôi - 23127396

Repository: linhkhoi1309/SpecReel: Spec-cited product video scripts with a claim checker for Vietnamese e-commerce sellers | Version: 1.0 | Date: 18/10/2026

1. Problem and users

Primary user: Hoàng Nam, owner of a mobile accessories and small electronics shop (180 active SKUs) on Shopee and TikTok Shop, at the moment a new product is listed and needs a short video ad before it goes live.

Problem (one sentence): When a new product is listed, the seller needs a 30-second video ad, but scripting and editing one by hand takes 1 to 1.5 hours per SKU and general AI writers add claims the spec does not support, so products go live without video or with claims that cause returns and listing takedowns.

What they do today: He cross-references the supplier's technical sheet (wattage, charging protocols, cable materials) to write a script in Notion, then manually syncs product photos and text overlays in a CapCut template, which costs him 10 to 12 hours per week across 8–10 new gadgets.

Evidence: Interview with 2 sellers on 18/10/2026; 40 public listings will form the eval set. Key quote: *"ChatGPT wrote that our 20W charger supports 'Samsung 45W Super Fast Charging 2.0' and is 'fully waterproof' when the spec only said IPX4 splash-resistant—three buyers returned it in the first week and left 1-star reviews."*

Why now: An LLM can write a natural script from a free-text listing and judge whether each sentence follows from the spec, including paraphrased and implied claims that keyword rules miss.

2. The LLM feature and the cost of being wrong

Feature: The seller imports one listing (title, description, spec table, photos) -> the LLM writes a 30-second script and shot list where every factual sentence cites a spec field, and a claim checker flags unsupported or prohibited claims -> the seller fixes or approves the script, and only then is the video rendered.

Concrete example: Input: a 20W charger listing whose spec says "20W output, IPX4 splash-resistant". Expected output: the line "20W charging, splash-resistant to IPX4" citing power and ip_rating; the generated lines "supports Samsung 45W Super Fast Charging 2.0" and "fully waterproof" are blocked as unsupported by the spec.

Why an LLM: Writing the script is generation, and checking it is entailment: "charges your phone quickly" can follow from "20W output", but "45W Super Fast Charging 2.0" and "waterproof" do not follow from "20W, IPX4". That judgement depends on meaning, which keyword lists and regex cannot capture.

Cost of a wrong answer:

Failure mode	Who is affected	Cost (quantified)	Can it be undone? How?
Unsupported or prohibited claim reaches the video	Buyers; the seller	Each complaint costs a refund of the order value plus return shipping ~350,000 VND ($14); repeated violations risk listing takedown	No, once posted. Prevented by the approval gate before render; after posting the video can only be removed
Missed output: a true claim is blocked, or a key spec (size, variant) is left out	The seller	About 5 minutes of manual editing per video (our estimate); a weaker ad	Yes: the seller edits the script before approving, within minutes
Confident but unsupported: a citation points to a spec field that does not say it	The seller, who trusts the citation; buyers	Same as the first row, plus lost trust in every citation	Only before approval: the cited field is shown next to each sentence, and code checks that the field exists

How we would know: A labelled set where each claim is tagged supported, unsupported or prohibited; every claim the seller deletes after approval is logged as a miss; a weekly audit of 20 approved scripts against their specs.

Guardrails and human-in-the-loop: Rendering starts only after the seller approves the script. A sentence without a valid spec citation is blocked by code, not by the model. Claims the checker is unsure about are marked for review, never passed automatically. Listing text is treated as data, never as instructions, with prompt-injection cases in the test set.

Success metric (evaluated in PA#4): Recall of unsupported or prohibited claims >= 90% and citation precision >= 95% on a labelled test set of 40 real listings; critical errors (a prohibited claim in an approved script) = 0.

3. Scope: in and out

Core hypothesis for this semester: If the seller uses SpecReel for each new listing, then a video is ready to post after 10 minutes or less of their time with no unsupported claims, measured by time-to-approve and the PA#4 claim metrics.

In scope this semester (maps to plan)	Out of scope (and why)
Script and shot-list generation in Vietnamese with spec citations, plus the claim checker (CP3, CP4)	Generative text-to-video: costly per clip and non-deterministic; template rendering is enough to test the hypothesis
Listing import and script editor with approval gate (CP2, CP3, CP4)	Posting to Shopee or TikTok Shop through their APIs: needs platform app approval, so marketplace calls are mocked
Labelled eval set of 40 listings and a metric dashboard (CP4, CP5)	Music, multi-product videos and an in-browser video editor: not needed to test the hypothesis
Template render worker (photos, captions, voiceover)	Fine-tuning our own model, and scripts in languages

with MP4 export, seller and staff roles, Docker release with rollback (CP2, CP5)	other than Vietnamese: not needed to test the hypothesis

4. Plan and ownership

Roles: Lương Linh Khôi - product owner, backlog, evaluation and the LLM feature; Nguyễn Hoàng Gia Bảo - backend, data model, render worker and release.

Six checkpoints (CP1 to CP5 dates from the course calendar; CP6 date to be confirmed with the lecturer):

Checkpoint	Date	Work / deliverable	Owner	Done when
CP1 - Proposal (PA#1)	09/10	Proposal, repository, backlog, seller interview	Lương Linh Khôi	Proposal and SELF_ASSESSMENT_REPORT.md submitted
CP2 - Core flow (PA#2)	13/11	Data model with migrations; listing import API (idempotent, paginated, versioned); seller and staff roles with ownership checks; CI with contract tests	Nguyễn Hoàng Gia Bảo	CI green on main; contract and Playwright import tests pass
CP3 - LLM feature (PA#3)	27/11	Script generation streamed to the editor; spec citations through tool calling; retrieval over spec and claim rules	Lương Linh Khôi	A real listing produces a cited script in the UI
CP4 - Evaluation (PA#4)	04/12	Claim checker, 40-listing eval set, guardrails, approval gate, injection tests	Lương Linh Khôi	Eval report with measured recall and precision; render blocked without approval
CP5 - Operations (PA#5)	11/12	Render worker in Docker; canary of prompt versions with rollback; logs, metrics, traces	Nguyễn Hoàng Gia Bảo	Dashboard shows cost and grounding rate per video; a rollback is demonstrated
CP6 - Final demo	18/12	Release v1.0 with release notes; end-to-end showcase from listing import to rendered video; post-mortem of what changed since PA#1	Nguyễn Hoàng Gia Bảo	Tagged v1.0 deployed from Docker; live demo runs end to end on a real listing; post-mortem in the repository

Working agreement: backlog in GitHub Projects; weekly sync Saturday, 10:00; Definition of Done: merged through a reviewed pull request, CI green, demo-able.

5. Risks

Top two risks that would sink this project:

Risk	Early signal	Mitigation we start this week	Fallback if it happens
R1 - The claim checker misses unsupported claims below target	First 20-listing eval run gives recall below 80%	Khôi labels 20 listings and runs a baseline prompt by 26/10	Restrict claims to attribute-value pairs matched by code against the spec; keep human review mandatory
R2 - No real seller or realistic listings to test with	Neither seller replies to follow-up by 25/10, or fewer than 40 usable listings collected by 01/11	Khôi contacts sellers and collects public listings this week; Bảo builds the import format by 26/10	Use public listings only, labelled separately by both members, with agreement reported

6. Technology choices

Stack:

Layer	Choice	Why this, for this project	Cost
LLM provider / model	Google Gemini 3.8 Flash; Gemini 3.8 Flash-Lite TTS for voiceover	Structured output and function calling for cited scripts; free tier for development; TTS on the same API (Vietnamese voice tested in CP2)	Paid tier through 31/12/2026: $0.75 input / $3.75 output per 1M tokens; TTS $0.0015 per 10 s of audio (ai.google.dev/gemini-api/docs/pricing, read 21/10/2026) -> about $0.013 per video
Frontend	Next.js (React)	Streams the script into the editor as it is generated; Playwright tests the approve flow	Free (self-hosted)
Backend / API	FastAPI (Python) and a Remotion render worker (Node, separate Docker service)	OpenAPI schema from typed models drives contract tests; video templates are React components, the same skills as our Next.js frontend, and the shot list fills them as input props; frame-based rendering is deterministic, so snapshot frames can be golden-file tested	Free: Remotion's free license covers teams of up to 3 people

Data / storage	PostgreSQL with full-text search, Alembic migrations, MinIO for media	One database for listings, claims and rule retrieval; migrations practised in CP2; S3-compatible storage runs in Docker	Free (Docker)
Evaluation / monitoring	pytest eval harness in GitHub Actions; OpenTelemetry and Grafana	Runs the labelled set on every prompt change and blocks merges below target; traces link each claim to its LLM call	Free (Actions free minutes, self-hosted Grafana)

Cost estimate: about 2,000 runs over the semester (development, eval runs, demos) x 2 LLM calls per run (about 6,000 input and 1,100 output tokens in total, our assumption) at Gemini 3.8 Flash prices, plus 30 s of TTS = about $0.013 per run, about $26 for the semester. Budget cap: $30.